

UT-07: UTILIZACIÓN AVANZADA DE CLASES

¿Qué vamos a ver?

- HERENCIA.
 - CONCEPTO Y TIPOS (SIMPLE Y MÚLTIPLE).
 - SUPERCLASES Y SUBCLASES.
 - CONSTRUCTORES Y HERENCIA.
- POLIMORFISMO.
 - CONCEPTO.
 - POLIMORFISMO EN TIEMPO DE COMPILACIÓN
 - POLIMORFISMO EN TIEMPO DE EJECUCIÓN (LIGADURA DINÁMICA).
- CLASES Y MÉTODOS ABSTRACTOS Y FINALES.
- INTERFACES. CLASES ABSTRACTAS VS. INTERFACES.
- COMPROBACIÓN ESTÁTICA Y DINÁMICA DE TIPOS.
- CONVERSIONES DE TIPOS ENTRE OBJETOS (CASTING).
- CLASES Y TIPOS GENÉRICOS O PARAMETRIZADOS.
- ENUMERADOS

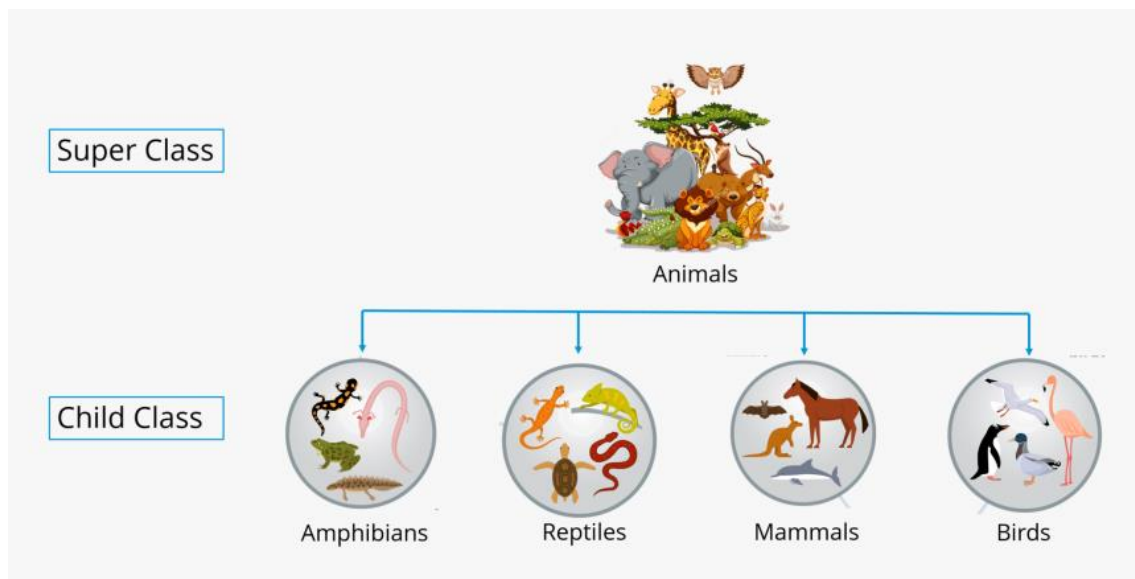
1. HERENCIA

1.1. Concepto y tipos

La herencia es la propiedad que permite a los objetos ser contruidos a partir de otros objetos. La idea fundamental es permitir crear nuevas clases aprovechando las características (atributos y métodos) de otras clases ya creadas evitando así tener que volver a definir esas características (reutilización).

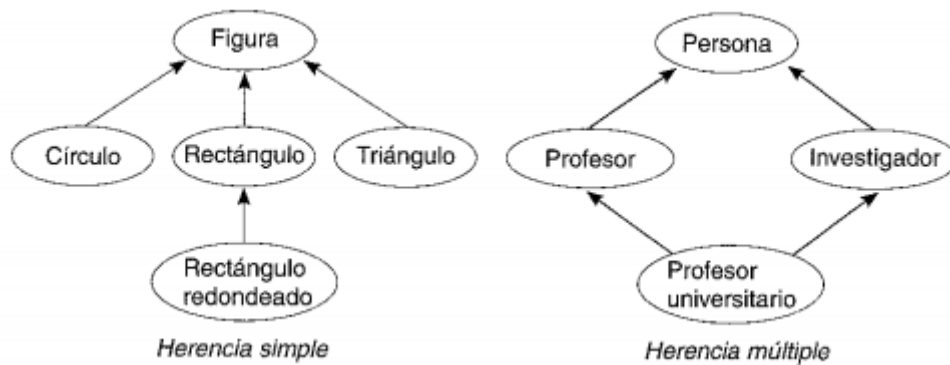
A una clase que hereda de otra se le llama subclase (clase derivada, clase hija) y aquella de la que se hereda es conocida como superclase (clase base, clase padre).

Las clases derivadas heredan el código y datos de su clase base, añadiendo su propio código y datos, incluso puede cambiar aquellos elementos de la clase base que necesita que sean diferentes. Por ejemplo, la clase animal se puede dividir en las subclases anfibios, mamíferos, aves, etc., y la clase vehículo en coches, camiones, etc.



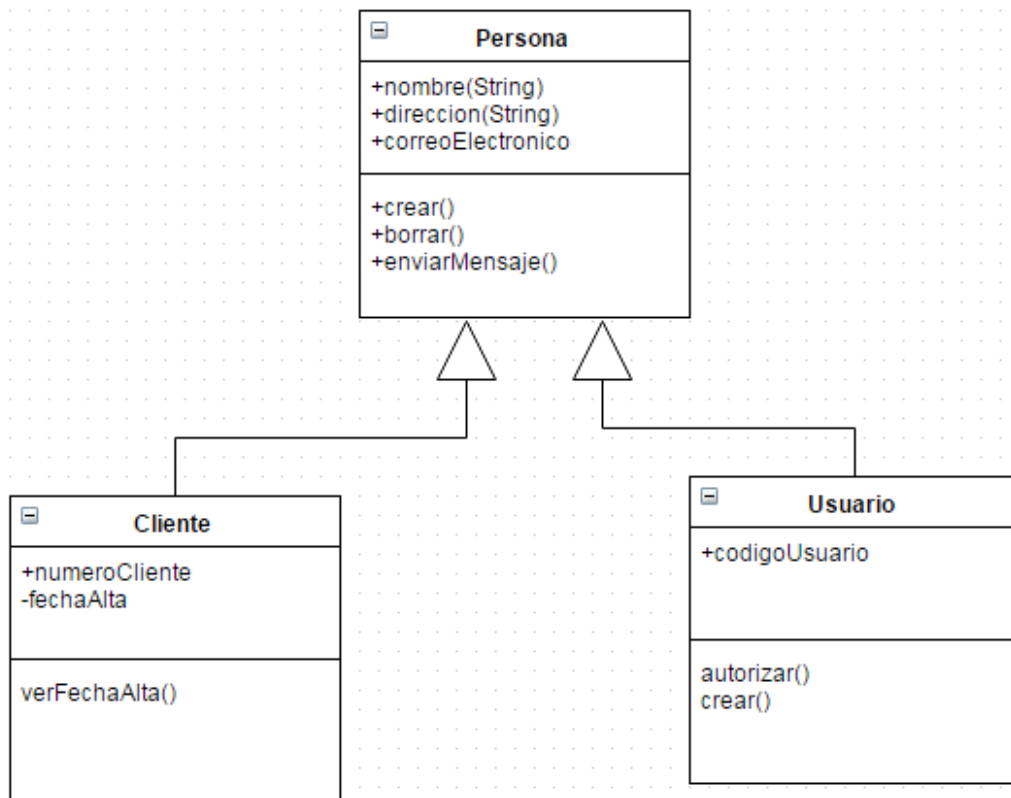
Existen dos tipos de herencia:

- **Herencia simple:** Una subclase puede heredar datos y métodos de una única clase. Java admite solo herencia simple.
- **Herencia múltiple:** Una subclase puede heredar datos y métodos de más de una clase. C++ admite herencia simple y múltiple.



La herencia tiene como ventajas principales la reutilización y compartición de código (evita repetir código ahorrando mucho tiempo), la consistencia de la interfaz (el comportamiento que heredan será el mismo en todos los casos) y la ocultación de información. Pero también tiene inconvenientes como son la velocidad de ejecución (los métodos heredados suelen ser más lentos) y la curva de aprendizaje (al principio el aprendizaje de la programación orientada a objetos suele ser más lento).

1.2. Superclases y subclases



Para que una clase (subclase) herede de otra clase (superclase) utilizamos la palabra clave “extends” seguida del nombre de la superclase.

```
public class Dog extends Animal{
    ...
};
```

Una subclase que hereda de una superclase adquiere los atributos y los métodos de la segunda, y además puede añadir unos propios.

Veamos un ejemplo...

```
class Animal {  
  
    // fields of the parent class  
    String name;  
    String species;  
    double weight;  
    double height;  
  
    ...  
  
    // method of the parent class  
    public feed(String food){  
        System.out.println("I eat " + food);  
    }  
}  
  
class Dog extends Animal {  
    String color;  
    String breed;  
  
    ...  
  
    public void display() {  
        System.out.println("My name is " + name);  
        System.out.println("My species is " + species);  
        System.out.println("My weight is " + weight);  
        System.out.println("My height is " + height);  
        System.out.println("My color is " + color);  
        System.out.println("My breed is " + breed);  
    }  
}
```

En este caso la clase Dog hereda de la clase Animal, por tanto, la clase Dog, además de sus atributos color y breed tiene los atributos, name, species, weight y height. Por otro lado, además del método display() cuenta con el método feed().

```
Dog dog1 = new Dog();  
...  
dog1.feed("meat");  
System.out.println("The name of my dog is: " + dog1.getName());
```

1.3. Constructores y herencia

Al igual que ocurre con el resto de métodos, los constructores también se heredan pero de una forma especial al resto de métodos.

Al llamar al constructor de una subclase vamos a tener que llamar al constructor de la superclase y si necesitamos alguna operación adicional específica de la subclase la realizaremos a continuación. Vamos a verlo con el ejemplo anterior:

```
class Animal {  
  
    String name;  
    String species;  
    double weight;  
    double height;  
  
    public Animal (){}  
  
    public Animal (String name, String species, double weight, double  
height){  
        this.name = name;  
        this.species = species;  
        this.weight = weight;  
        this.height = height;  
    }  
  
    ...  
}  
  
class Dog extends Animal {  
    String color;  
    String breed;  
  
    public Dog(){  
        super();  
    }  
  
    public Dog(String name, String species, double weight, double height,  
String color, String breed){  
        super(name, species, weight, height);  
        this.color = color;  
        this.breed = breed;  
    }  
  
    ...  
}
```

1.4. Modificador de acceso `protected`

Cuando hablamos de herencia entre clases nos encontramos con un nuevo modificador de acceso, que está precisamente especializado en herencia de clases, este modificador es “`protected`”.

Este modificador funciona de una forma similar a la visibilidad por defecto, con la diferencia de que los miembros protegidos serán siempre visibles para las clases que hereden, independientemente de si la superclase y la subclase se encuentra en el mismo paquete o son externas, aunque en este último caso, habrá que importar la superclase.

En resumen, un miembro “`protected`” es visible en las clases del mismo paquete, no es visible para las clases externas, pero siempre es visible, independientemente del paquete al que pertenezca, desde una clase hija.

Modificador (palabra reservada)	Misma clase	Mismo paquete	Subclase de otro paquete	Resto
<i>private</i>				
(defecto)				
<i>protected</i>				
<i>public</i>				

```
class Person {
    protected String fname = "John";
    protected String lname = "Doe";
    protected String email = "john@doe.com";
    protected int age = 24;
}
```

```
class Student extends Person {  
    private int graduationYear = 2018;  
    public static void main(String[] args) {  
        Student myObj = new Student();  
        System.out.println("Name: " + myObj.fname + " " + myObj.lname);  
        System.out.println("Email: " + myObj.email);  
        System.out.println("Age: " + myObj.age);  
        System.out.println("Graduation Year: " + myObj.graduationYear);  
    }  
}
```

2. POLIMORFISMO

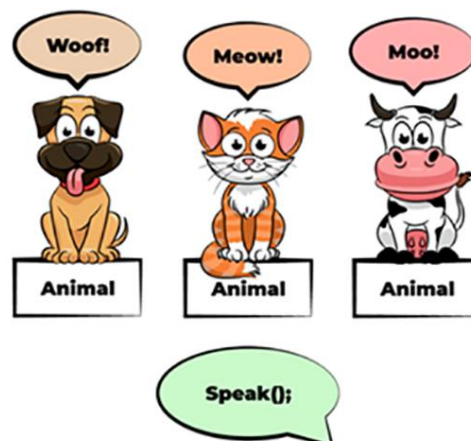
El polimorfismo es la propiedad que permite que un operador o un método actúen de modo diferente en función del objeto sobre el que se aplican.

Cuando un operador existente en el lenguaje tal como +, = o * se le asigna la posibilidad de operar sobre un nuevo tipo de datos, se dice que está sobrecargado. La sobrecarga es una clase polimorfismo.

En un sentido más general, el polimorfismo supone que un mismo mensaje puede producir acciones totalmente diferentes cuando se reciben por objetos diferentes.

El polimorfismo adquiere su máxima expresión en la herencia de clases, es decir, cuando se obtiene una clase a partir de una clase ya existente.

Por ejemplo, cuando se describe una clase base Mamífero se puede observar que la operación comer es una operación fundamental en su vida, de modo que cada tipo de mamífero (vaca, humano, león) debe poder realizar la operación comer, aunque cada una de las clases derivadas que representan los distintos tipos de mamíferos la realizará de un modo diferente (polimorfismo).



2.1. Polimorfismo en tiempo de compilación

El primer tipo de polimorfismo en tiempo de compilación ya lo vimos en la unidad anterior, se trata de la sobrecarga de métodos.

Además de la sobrecarga de métodos se nos está permitido redefinir los tipos de datos de los atributos heredados y de esta forma adaptar mejor dichos atributos a las clases hijas. Aquí tenemos un ejemplo:

```
class Persona {  
    String nombre;  
    byte edad;  
    double estatura;  
    void mostrarDatos(){  
        System.out.println(nombre);  
        System.out.println(edad);  
        System.out.println(estatura);  
    }  
}
```

```
//subclase Empleado que hereda de Persona  
class Empleado extends Persona {  
    String estatura; //cambiamos de double a String S, M, L, XL  
    double salario;
```

```
@Override
void mostrarDatos(){
    System.out.println(nombre);
    System.out.println(edad);
    System.out.println(estatura);
    System.out.println(salario);
}
}
```

Por otro lado, cuando una subclase sustituye la funcionalidad de un método de su superclase, se le denomina sobreescritura del método.

Para realizar esto en Java vamos a utilizar la anotación “@Override” antes de la definición del método de la subclase.

Veamos un ejemplo:

```
class Animal {
    void sound() {
        System.out.println('Animals make sounds');
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println('Dogs bark');
    }
}

// Create an instance of Dog
Dog dog = new Dog();
// Call the sound method
dog.sound();

// Output:
// 'Dogs bark'
```

2.2. Polimorfismo en tiempo de ejecución (ligadura dinámica)

Debemos tener en cuenta que un objeto de una subclase puede ser asignado a una variable de su superclase, por ejemplo, si tenemos una superclase Persona

y una subclase Empleado, podremos crear una variable de la clase Persona y asignarle un objeto que creamos de la clase Empleado:

```
public class Persona{
    protected String nombre;
    protected String apellido;
    protected String DNI;

    void mostrarInformacion(){
        System.out.println("Mi nombre es " + this.nombre);
        System.out.println("Mi apellido es " + this.apellido);
        System.out.println("Mi DNI es " + this.DNI);
    }
}

public class Empleado extends Persona{
    protected int sueldo;

    @Override
    void mostrarInformacion(){
        System.out.println("Mi nombre es " + this.nombre);
        System.out.println("Mi apellido es " + this.apellido);
        System.out.println("Mi DNI es " + this.DNI);
        System.out.println("Mi sueldo es " + this.sueldo);
    }
}
```

```
Empleado empleado = new Empleado(nombre, apellido, DNI);
Persona persona = empleado;
```

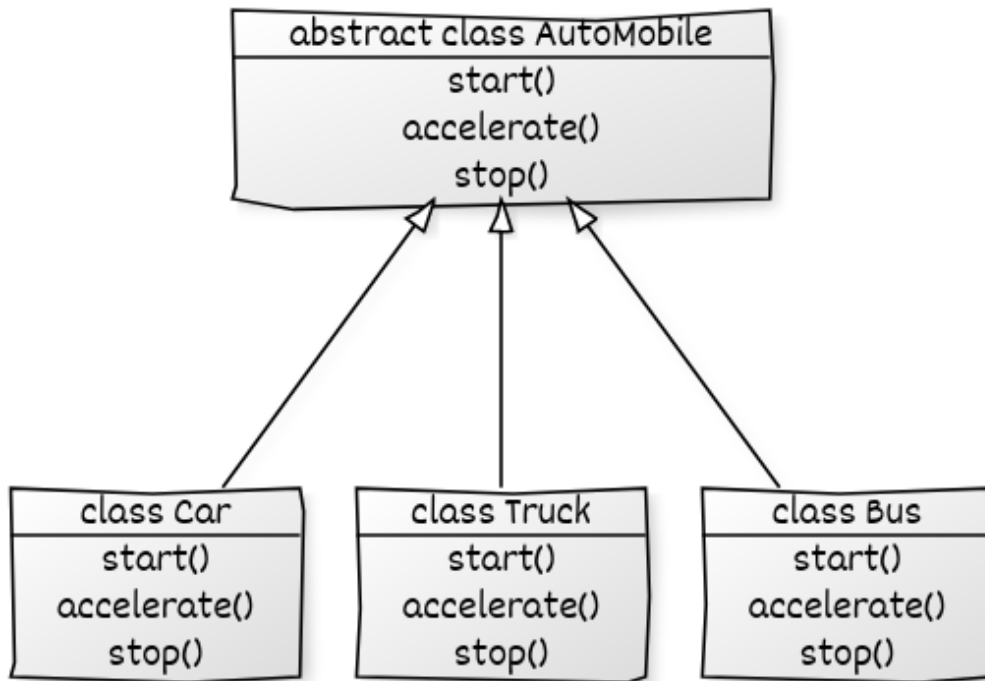
En este caso, si llamamos al método `mostrarInformación()` de `empleado`, nos devolverá toda la información, incluido el sueldo, pero si llamamos a `mostrarInformacion()` de `persona`, solo nos mostrará el nombre, apellido y DNI.

```
empleado.mostrarInformacion(); //nombre, apellido, DNI y sueldo
persona.mostrarInformacion(); //nombre, apellido y DNI
```

A esto se le denomina ligadura dinámica y nos sirve para poder acceder tanto a la versión de la superclase del método, como a la versión de la subclase.

3. CLASES Y MÉTODOS ABSTRACTOS Y FINALES

3.1. Clases y métodos abstractos



En la jerarquía de herencia de clases, cuanto más abajo, más específica y particular es la implementación de los métodos. Asimismo, cuanto más arriba, más general.

Hay métodos que no podemos implementar en una clase determinada por falta de datos, pero sí en sus subclases, donde se han añadido los atributos necesarios. La idea es implementarlos “vacíos”, solo con la declaración, en la superclase, y hacen *overriding* en las subclases, donde ya disponemos de la información necesaria para implementar los detalles.

Un método declarado en una clase, pero cuya implementación se delega a las subclases, se conoce como **método abstracto**. Para declarar un método abstracto se le antepone el modificador “abstract” y se declara un prototipo, sin escribir el cuerpo de la función. Por ejemplo, para declarar un método abstracto que muestra información del objeto escribiremos:

```
abstract void mostrarDatos();
```

Las subclases deberán implementar el método `mostrarDatos()`, cada una con las particularidades específicas de la clase, que no se conocen al nivel de la superclase.

Toda clase que tiene un método abstracto debe ser declarada, a su vez, abstracta, también utilizando la palabra clave "abstract".

Las clases abstractas no son instanciables, es decir, no se pueden crear objetos de esa clase. Las clases abstractas existen para ser heredadas por otras, y no para ser instanciadas.

Una clase abstracta puede tener tanto métodos abstractos como no abstractos, e incluso algunos atributos declarados, pero siempre tendrá al menos un método abstracto.

```
abstract class Persona{
    protected String nombre;
    protected String apellido;
    protected String DNI;

    abstract void mostrarInformacion();

    public void hablar(String mensaje){
        System.out.println(mensaje);
    }
}

public class Empleado extends Persona{
    protected int sueldo;

    @Override
    void mostrarInformacion(){
        System.out.println("Mi nombre es " + this.nombre);
        System.out.println("Mi apellido es " + this.apellido);
        System.out.println("Mi DNI es " + this.DNI);
        System.out.println("Mi sueldo es " + this.sueldo);
    }
}
```

3.2. Clases y métodos finales

Existirán ocasiones en las que no deseemos que una de nuestras clases pueda ser heredada. Para poder conseguir esto vamos a colocar el modificador “final” a la clase:

```
public final class Persona{
    protected String nombre;
    protected String apellido;
    protected String DNI;

    void mostrarInformacion(){
        System.out.println("Mi nombre es " + this.nombre);
        System.out.println("Mi apellido es " + this.apellido);
        System.out.println("Mi DNI es " + this.DNI);
    }
}
```

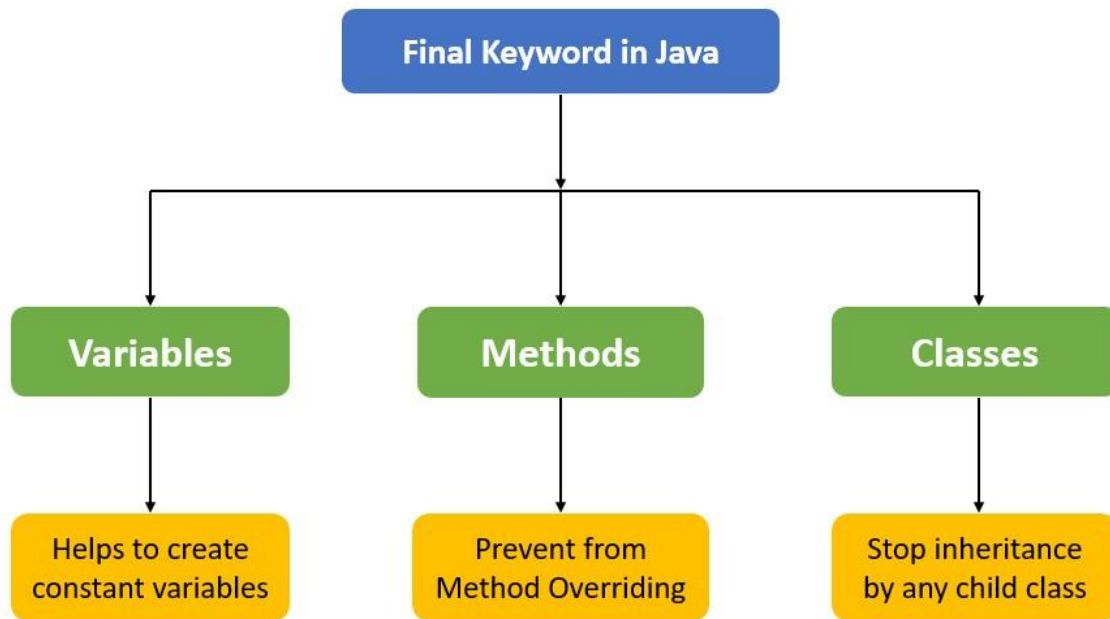
De esta forma nadie podrá crear subclases de la clase Persona.

De la misma manera, podemos considerar que alguno de los métodos de nuestra superclase no de ser sobre escrito por una de sus subclases. Al igual que ocurre con las clases, para evitar esto vamos a añadir el modificador final a la declaración de nuestro método.

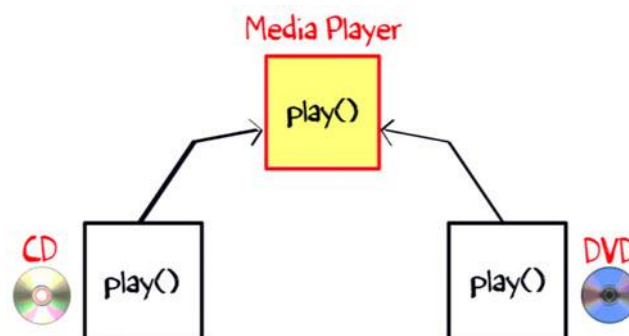
```
public class Persona{
    protected String nombre;
    protected String apellido;
    protected String DNI;

    final void mostrarInformacion(){
        System.out.println("Mi nombre es " + this.nombre);
        System.out.println("Mi apellido es " + this.apellido);
        System.out.println("Mi DNI es " + this.DNI);
    }
}
```

Así, aunque alguien cree una subclase de Persona no le será posible sobrescribir el método mostrarInformacion().



4. INTERFACES



Las interfaces son una especie de plantilla para la construcción de clases. Normalmente una interfaz se compone de un conjunto de declaraciones de cabeceras de métodos (sin implementar, igual que los métodos abstractos) que especifican un protocolo de comportamiento para una o varias clases.

Además, una clase puede implementar una o varias interfaces. En ese caso, la clase debe proporcionar la declaración y definición de todos los métodos de cada una de las interfaces o bien declararse como clase abstracta.

Por otro lado, una interfaz puede emplearse también para declarar constantes que luego puedan ser utilizadas por otras clases.

Es decir, se trata de una clase que no puede ser implementada por si misma, sino que otras clases la heredan y la implementan. De este modo, al emplear las interfaces, es posible establecer un conjunto de reglas que otras clases deberán seguir de forma estricta.

Una interfaz puede contener:

- Constantes estáticas.
- Definición de métodos abstractos.
- Implementación de métodos por defecto.
- Implementación de métodos estáticos.
- Implementación de métodos privados.

4.1. Definición de una interfaz

Para definir una interfaz únicamente tendremos que utilizar la palabra clave "interface" seguido del nombre de la interfaz. Por consenso la primera letra del nombre de la interfaz es una "I".

```
public interface IAnimal {  
  
}
```

4.2. Constantes estáticas

El único tipo de atributos que se pueden incluir en una interfaz son constantes estáticas, aunque no es necesario identificarlos como "static" y "final", ya que se toma por defecto.

```
public interface IAnimal {  
    String PLANETA = "Tierra";  
}
```

Estas constantes serán heredadas y podrán ser utilizadas por las instancias de las clases que implementen dicha interfaz.

También será posible llamar a estos atributos desde la interfaz no implementada.

```
System.out.println("Los animales viven en el planeta" + IAnimal.PLANETA);
```


4.3. Declaración de métodos abstractos

Lo más habitual es que utilicemos las interfaces para declarar métodos abstractos que luego tendrán que implementar las clases que las instancien.

La forma de declarar estos métodos será similar a la forma en que lo hacemos en las clases abstractas, con la diferencia de que en este caso no será necesario utilizar las palabras claves “abstract” ni “public” ya que se asumen por defecto abstractos y públicos.

```
public interface IAnimal {  
    String PLANETA = "Tierra";  
  
    void alimentar(float cantidaComida);  
    void desplazar();  
    float velocidad();  
}
```

4.4. Implementación de métodos por defecto

Los métodos por defecto se declaran anteponiendo la palabra reservada “default” y son “public” sin necesidad de especificarlos.

Estos métodos serán incorporados a las clases que implementen el interfaz y serán accesibles para todos los objetos de dichas clases.

Además, las clases que implementan la interfaz podrán sobrescribir estos métodos.

```
public interface IAnimal {  
    String PLANETA = "Tierra";  
  
    void alimentar(float cantidaComida);  
    void desplazar();  
    float velocidad();  
  
    default void hacerRuido(){  
        System.out.println("Shhhhh");  
    }  
}
```

4.5. Implementación de métodos estáticos

En una interfaz también se pueden implementar métodos estáticos. Si no se especifica un modificador de acceso se tomarán como “public”. Estos métodos pertenecen a la interfaz, y no a las clases que la implementan, por tanto, este método será accesible directamente desde la interfaz.

```
public interface IAnimal {  
    String PLANETA = "Tierra";  
  
    void alimentar(float cantidaComida);  
    void desplazar();  
    float velocidad();  
  
    static int calcularEdad(int anioNacimiento){  
        return LocalDate.now().getYear() - anioNacimiento;  
    }  
}
```

Para llamar a la función lo haríamos así...

```
int edad = IAnimal.calcularEdad(1999);
```

4.6. Implementación de métodos privados

Además de los métodos abstractos, por defecto y estáticos, que son públicos por defecto, en una interfaz se pueden implementar métodos privados anteponiendo el modificador private. Pueden ser tanto estáticos como no estáticos y no son accesibles fuera del código de la interfaz. Estos métodos están implementados y solo pueden ser invocados por el resto de métodos no abstractos de la interfaz. En general, son métodos auxiliares para uso interno de otros métodos del interfaz.

```
public interface IAnimal {  
    String PLANETA = "Tierra";  
  
    void alimentar(float cantidaComida);  
    void desplazar();  
    float velocidad();
```

```
static int calcularEdad(int anioNacimiento){  
    return getAnioActual() - anioNacimiento;  
}  
  
private static int getAnioActual(){  
    return LocalDate.now().getYear();  
}  
}
```

4.7. Implementación de interfaces

Una vez definida nuestra interfaz, para que una clase la implemente debe utilizar la palabra clave “implements” y además estará obligado a implementar todos los métodos abstractos de la interfaz, a no ser que se trate de una clase abstracta.

```
public class Perro implements IAnimal {  
    private float VELOCIDAD = 30;  
  
    @Override  
    public void alimentar(float cantidadComida) {  
        System.out.println("Como " + cantidadComida + " kilos de carne al  
día");  
    }  
  
    @Override  
    public void desplazar() {  
        System.out.println("Me desplazo a cuatro patas");  
    }  
  
    @Override  
    public float velocidad() {  
        return VELOCIDAD;  
    }  
}
```

Como ya hemos comentado antes una clase puede implementar varias interfaces simultáneamente. En este caso deberíamos implementar los métodos abstractos de todas las interfaces que implementa.

```
public class Perro implements IAnimal, IMamal {  
    ...  
}
```

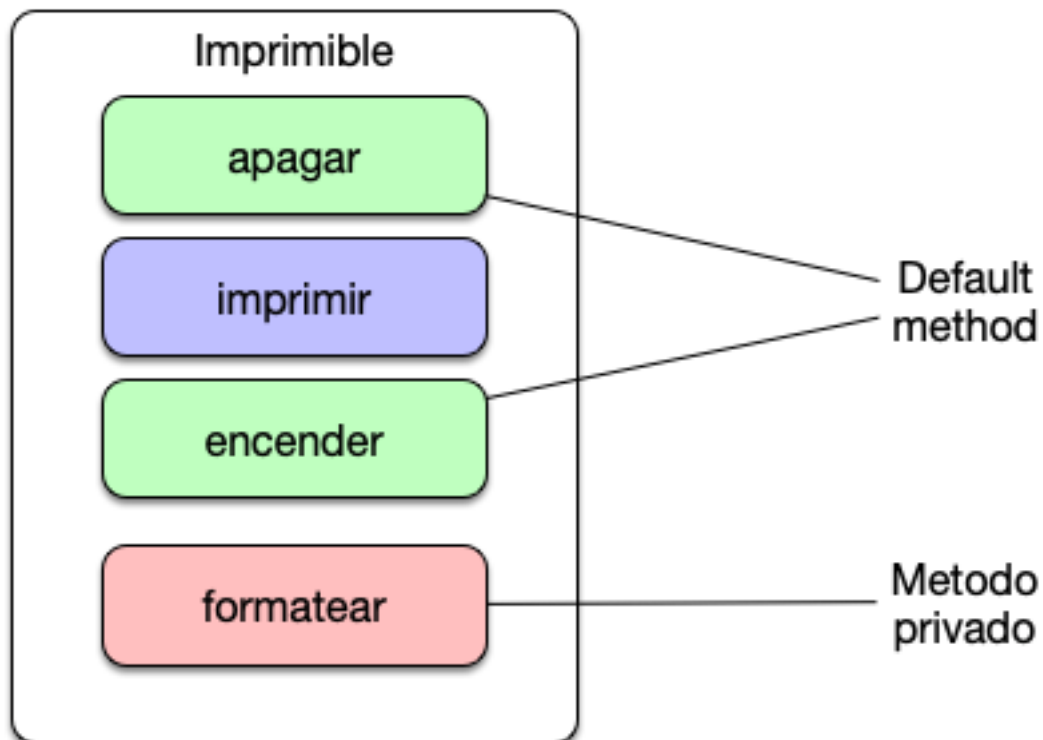
También existe la posibilidad de que una clase herede una clase e implemente una o varias interfaces a la vez.

```
public class Perro extends AnimalTerrestre implements IAnimal, IMamal {  
    ...  
}
```

Y, finalmente, también existe la posibilidad de que una interfaz herede de otra.

```
public interface IMamal extends IAnimal {  
    ...  
}
```

Esto provocaría que la interfaz que estamos definiendo heredaría todos los atributos y todos los métodos, a excepción de los privados de la interfaz madre.



4.8. Clases abstractas vs interfaces

Una interfaz puede parecer similar a una clase abstracta, pero existen una serie de diferencias entre ellas:

- Una interfaz no declara variables de instancia (atributos)
- Una clase puede implementar varias interfaces, pero sólo puede heredar de otra clase
- Una clase abstracta pertenece a una jerarquía de clases, mientras que una interfaz no pertenece a una jerarquía de clases. En consecuencia, clases sin relación de herencia pueden implementar la misma interfaz.

Aunque depende de los casos, habitualmente es más conveniente utilizar interfaces que clases abstractas, debido a que nos estamos tan condicionados por la jerarquía y además nos permite implementación múltiple de interfaces por parte de una clase, mientras que las clases solo puede heredar de una única clase padre.

Abstract Class		Interface
1. abstract keyword	•	1. interface keyword
2. Subclasses extends abstract class	•	2. Subclasses implements interfaces
3. Abstract class can have implemented methods and 0 or more abstract methods	•	3. Java 8 onwards, Interfaces can have default and static methods
4. We can extend only one abstract class	•	4. We can implement multiple interfaces



4.9. Variables de tipo interfaz

Igual que ocurre con las subclases y las superclases, es posible que una situación determinada nos interese crear una variable de tipo interfaz al que pasaremos como valor un objeto de una clase que implemente dicha interfaz. Esto nos puede servir, entre otras cosas, para ocultar funcionalidad propia de la clase que no deseamos que se utilice desde fuera del objeto.

```
IAnimal animal = new Perro();
```

5. COMPARACIÓN ESTÁTICA Y DINÁMICA DE TIPOS



Existen muchas interfaces de gran importancia incluidas en la API de Java. Dos de ellas nos permiten realizar la comparación de valores y objetos, que podremos utilizar para realizar operaciones de búsqueda u ordenación, por ejemplo.

5.1. Interfaz Comparable

Cuando queremos establecer un criterio de comparación natural o por defecto entre los objetos de una clase, haremos que implemente la interfaz Comparable, que consta de un único método abstracto.

```
int compareTo(Object ob);
```

Este método será el encargado de establecer un criterio de ordenación entre los objetos de una clase. Para comparar dos objetos de una determinada clase que implemente Comparable, escribiremos:

```
obj1.comparteTo(obj2);
```

que devolverá un número entero.

Si en una ordenación el objeto obj1 debe ir antes que el objeto obj2, el método devolverá un número negativo; si debe ir después, devolverá un número positivo, y si deben ser iguales a efectos de ordenación, devolverá un cero.

Veamos un ejemplo:

```
public class Automovil implements Comparable {
    private float maxVelocidad;
    private String marca;
    private String modelo;

    public Automovil(float maxVelocidad, String marca, String modelo){
        this.maxVelocidad = maxVelocidad;
        this.marca = marca;
        this.modelo = modelo;
    }

    @Override
    public int compareTo(Object o) {
        Automovil otroAutomovil = (Automovil) o;
        if(maxVelocidad < otroAutomovil.maxVelocidad){
            return -1;
        }
        else if(maxVelocidad > otroAutomovil.maxVelocidad){
            return 1;
        }
        else{
            return 0;
        }
    }
}
```

```
Automovil coche1 = new Automovil(220, "Audi", "A3");
Automovil coche2 = new Automovil(150, "Ford", "Fiesta");

if(coche1.compareTo(coche2) > 0){
    System.out.println("Coche1 es más rápido");
}
else{
    System.out.println("Coche2 es más rápido");
}
```

Esto podemos utilizarlo para ordenar arrays de objetos, por ejemplo:

```
Automovil[] automoviles = new Automovil[]{
    new Automovil(220, "Audi", "A3"),
    new Automovil(150, "Ford", "Fiesta"),
    new Automovil(300, "Ferrari", "F40")};

Arrays.sort(automoviles);
//Nos ordenará el array así: Ford Fiesta, Audi A3, Ferrari F40
```

5.2. Interfaz Comparator

La interfaz Comparable proporciona un criterio de ordenación natural, que es el que usan por defecto los distintos métodos de la API de Java, como `sort()`. Pero es frecuente que tengamos que ordenar los objetos de la misma clase con distintos criterios en el mismo programa. Para resolver estos casos existe la interfaz `Comparator`, definida también en la API. Antes de usarla habrá que importarla con la sentencia:

```
import java.util.Comparator;
```

Esta interfaz tiene un único método abstracto:

```
int compare(Object obj1, Object obj2);
```

Recibe como parámetros dos objetos que queremos comparar para determinar cuál va antes y cuál después en un proceso de ordenación. Devuelve un entero, que será negativo si `obj1` va antes que `obj2`, positivo si va después y cero si son iguales. Llamamos comparador a cualquier objeto de una clase que implemente la interfaz `Comparator`.

Necesitaremos una clase de comparadores distinta por cada criterio de comparación que queramos emplear con objetos de una clase determinada. Pero, a diferencia de la interfaz Comparable, Comparator no se implementa en la clase de los objetos que queremos ordenar, sino en una clase específica, cuyos objetos llamaremos comparadores.

Veamos un ejemplo:

```
public class ComparadorCoches implements Comparator {  
  
    @Override  
    public int compare(Object o1, Object o2) {  
        Automovil automovil1 = (Automovil) o1;  
        Automovil automovil2 = (Automovil) o2;  
  
        if(automovil1.getMaxVelocidad() < automovil2.getMaxVelocidad()){  
            return -1;  
        }  
        else if(automovil1.getMaxVelocidad() >  
automovil2.getMaxVelocidad()){  
            return 1;  
        }  
        else{  
            return 0;  
        }  
    }  
}
```

```
ComparadorCoche comparadorCoche = new ComparadorCoche();  
  
if(comparadorCoches(coche1, coche2)){  
    System.out.println("Coche1 es más rápido");  
}  
else{  
    System.out.println("Coche2 es más rápido");  
}
```

Estas clases también las podremos utilizar para ordenar arrays, pero en esta ocasión debemos poner el objeto del comparador como segundo parámetro.

```
Automovil[] automoviles = new Automovil[]{  
    new Automovil(220, "Audi", "A3"),  
    new Automovil(150, "Ford", "Fiesta"),  
    new Automovil(300, "Ferrari", "F40")};
```

```
Arrays.sort(automóviles, comparadorCoches);
//Nos ordenará el array así: Ford Fiesta, Audi A3, Ferrari F40
```

Comparable	Comparator
An object comparing itself with other object.	Compares two objects.
Class itself must implement Comparable interface.	External Class implements the Comparator Interface.
Can be used to sort according to only one property.	Can be used to sort according to multiple properties.

6. CONVERSIONES DE TIPOS ENTRE OBJETOS

Es habitual que, en un array, u otra de las estructuras de almacenamiento en memoria que veremos más adelante, guardemos elementos que corresponden a la misma superclase, pero a subclases distintas. Como por ejemplo en siguiente caso... Imaginemos que Vendedor y JefeSeccion son subclases de Empleado:

```
Empleado[] empleados = new Empleado[3];

empleado[0] = new Vendedor(1, "James", "654789032");
empleado[1] = new Vendedor(2, "Steve", "656889459");
empleado[2] = new JefeSeccion(3, "Mark", "667739723");
```

Como vimos en la sección sobre ligadura dinámica, un objeto de una subclase también pertenece a la superclase de la que hereda, y por ello es posible que en un mismo array puedan convivir objetos de distinta clase, pero misma superclase.

Ahora supongamos que cada una de las supclases tiene un método propio que no comparte con las otras...

```
public class Vendedor extends Empleado{
    ...

    public double calcularComision(double importe){
        ...
    }
}

public class JefeSeccion extends Empleado{
    ...

    public void generarHorarioTurnos(){
        ...
    }
}
```

En este caso la clase Vendedor cuenta el método propio calcularComision y la clase JefeSeccion con el método propio generarHorarioTurnos.

Si nosotros intentásemos, por ejemplo, acceder al método calcularComision de uno de los Vendedores almacenados en el array, Java no nos lo permitiría, ya que, como vimos en la sección del enlace dinámico mientras que el objeto esté identificado como un objeto de la superclase, no tendrá acceso a las clases propias de las subclases, es decir, no podríamos hacer lo siguiente:

```
empleado[1].calcularComision(100);
```

Sino que para ello necesitaríamos realizar un casting del objeto de la superclase a la subclase. Para ello vamos a utilizar algo que ya hicimos con los valores numéricos... vamos a colocar el tipo de la subclase entre paréntesis delante del objeto...

```
Vendedor vendedor1 = (Vendedor) empleado[1];
vendedor1.calcularComision(100);
```

Llevando a cabo este casting, o conversión de tipo entre clases, ya nos será posible acceder al método que necesitemos de la subclase.

Existe un operador que nos va a ser de gran utilidad a la hora de identificar si un objeto dentro de un array de la superclase es de una subclase u otra. Este operador es "instanceof". Veamos un ejemplo práctico para entenderlo mejor.

Cogemos el array del ejemplo anterior, queremos recorrerlo y que cuando nos encontremos con un Vendedor calcule la comisión de una venta de 100€ y cuando se trate de un JefeSeccion, que genere el horario de turnos de su sección.

```
Empleado[] empleados = new Empleado[3];

empleado[0] = new Vendedor(1, "James", "654789032");
empleado[1] = new Vendedor(2, "Steve", "656889459");
empleado[2] = new JefeSeccion(3, "Mark", "667739723");
```

A primera vista sería imposible distinguir unos de otros, pero gracias a instanceof va a ser posible de la siguiente manera:

```
Vendedor vendedor;
JefeSeccion jefeSeccion;

for(Empleado empleado : empleados){
    if(empleado instanceof Vendedor){
        vendedor = (Vendedor)empleado;
        vendedor.calcularComision(100);
    }
    if(empleado instanceof JefeSeccion){
        jefeSeccion = (JefeSeccion)empleado;
        jefeSeccion.generarHorarioTurnos();
    }
}
```

Como veis, instanceof nos devolverá un true o false en función de si la clase del objeto coincide con la que estamos comparando, y va a ser el encargado de determinar si el empleado que hemos seleccionado es un Vendedor o un

JefeSeccion, y una vez identificado podemos realizar el casting y utilizar los métodos o atributos propios de esa subclase.

7. CLASES Y TIPOS GENÉRICOS O PARAMETRIZADOS

Los genéricos son clases, o tipos, que tienen un parámetro. Al crear una clase genérica, se especifica no solo la clase, sino también el tipo de dato con el que funcionará.

Un ejemplo muy simple de definición de clase genérica sería el siguiente:

```
public class Box<T> {  
    // T es el "tipo"  
    private T t;  
  
    public void set(T t) { this.t = t; }  
    public T get() { return t; }  
}
```

Se suele utilizar la letra T mayúscula como indicador de que ahí debe incluirse una clase a la hora de instanciar la clase, pero se podría utilizar cualquier otra letra.

Si te fijas, cuando tenemos que utilizar el tipo de dato que corresponde al que hemos pasado como parámetro a la clase, utilizaremos la misma letra para que el compilador identifique que se trata de la misma clase, o tipo de datos.

Vamos a intentar entender para qué sirven las clases genéricas con el siguiente ejemplo...

Cuando nosotros instanciamos un objeto del tipo Box, podremos pasarle cualquier clase como parámetro, por ejemplo:

```
Box<Persona> cajaPersona = new Box<Persona>();
```

De esta forma, podríamos decir, que todas las partes del código donde pone "T" se sustituiría por Persona. Algo así...

```
public class Box<Persona> {  
    private Persona persona;  
  
    public void set(Persona persona) { this.persona = persona; }  
    public Persona get() { return persona; }  
}
```

Pero ¿qué pasaría si en lugar de personas quiero guardar animales en mi clase Box? Pues solo tendríamos que hacer lo siguiente:

```
Box<Animal> cajaAnimal = new Box<Animal>();
```

Y Java identificaría que lo que queremos hacer es esto:

```
public class Box<Animal> {  
    private Animal animal;  
  
    public void set(Animal animal) { this.animal = animal; }  
    public Animal get() { return animal; }  
}
```

Con todo lo que hemos aprendido hasta ahora, si quisiésemos tener una clase Box que almacenara Personas y Animales, estaríamos obligados a crear dos clases distintas, y, por tanto, repetir código. Así:

```
public class BoxPersona {  
    private Persona persona;  
  
    public void set(Persona persona) { this.persona = persona; }  
    public Persona get() { return persona; }  
}  
  
public class BoxAnimal {  
    private Animal animal;  
  
    public void set(Animal animal) { this.animal = animal; }  
    public Animal get() { return animal; }  
}
```

Pero gracias a las clases genéricas podemos utilizar el mismo código para una o para muchas clases distintas, únicamente variando el tipo que le pasamos por parámetro a la clase.

```
public class Box<T> {  
    // T es el "tipo"  
    private T t;  
  
    public void set(T t) { this.t = t; }  
    public T get() { return t; }  
}
```

Igual que con las clases, podemos crear interfaces genéricas:

```
public interface MyInterface<T> {  
    void doSomething(T value);  
}
```

En algunas ocasiones solo nos interesará que se pueden pasar por parámetro una clase y sus subclases, o una subclase y su superclase. Para ello utilizaremos los parámetros genéricos limitados.

En el caso de querer que solo puedan pasarse una clase y sus subclases, utilizaríamos un código similar a este:

```
class nombreClase<T extends claseLimite>{  
    ...  
}
```

Esto obligará a que cuando generemos una instancia de la clase debemos introducir por parámetro la clase claseLimite o alguna de sus subclases.

En el caso de querer que solo puedan pasarse una subclase y su superclase, utilizaríamos un código similar a este:

```
class nombreClase<T super claseLimite>{  
    ...  
}
```

Esto obligará a que, al crear una instancia de nombreClase, debemos introducir como parámetro la subclase claseLimite o su superclase.

También podemos aplicar estas limitaciones a clases que instancien un interfaz, y se haría de la misma forma que con las clases, utilizando la palabra “extends” en lugar de “implements”.

```
class nombreClase<T extends MiInterfaz>{  
    ...  
}
```

8. ENUMERADOS

Un enumerado es una clase “especial”, tanto en Java como en otros lenguajes, que limitan la creación de objetos a los especificados explícitamente en la implementación de la clase. La única limitación que tienen los enumerados respecto a una clase normal es que, si tienen constructor, este debe de ser privado para que no se puedan crear nuevos objetos.

En nuestro caso solo los vamos a utilizar para crear agrupaciones de constantes, pero con los enumerados se pueden hacer muchas más cosas. Os recomiendo que, si os interesa, investiguéis sobre el tema.

Para declarar un enumerador necesitaremos utilizar la palabra reservada “enum”, le daremos nombre y, entre llaves introduciremos los literales que deseamos utilizar.

```
public enum Demarcacion {  
    PORTERO, DEFENSA, CENTROCAMPISTA, DELANTERO  
}
```


Al tratarse de constantes, se recomienda poner los nombres de los literales en mayúsculas.

Empezando por el primero hasta el último se asigna de forma automática un valor, comenzando por 0, a cada uno de los literales. Es decir, en el ejemplo anterior PORTERO sería el 0, DEFENSA el 1, etc...

Ahora vemos las operaciones básicas que podemos utilizar con este tipo de enumerados.

Utilizar uno de los valores del enumerado:

```
Demarcacion.DELANTERO;
```

Asignar a una variable del tipo enumerado:

```
Demarcacion delantero = Demarcacion.DELANTERO;
```

Extraer el nombre correspondiente de una variable asignada un enumerado:

```
delantero.name(); //Devuelve un String con el nombre de la constante  
(DELANTERO)
```

Conocer la posición que ocupa, o el valor, del enumerado asignado:

```
delantero.ordinal(); // Devuelve un entero con la posición del enum según  
está declarado
```

Comparar dos variables del mismo tipo enum según su ordenación:

```
Demarcacion portero = Demarcacion.PORTERO;  
delantero.compareTo(portero); //Devolverá un valor superior a 0, ya que  
la posición de delantero es mayor que la de portero
```

Obtener un array con todos valores del enum:

```
Demarcacion.values(); //Devuelve un array que contiene todos los enum
```

Veamos el ejemplo completo:

```

public enum Demarcacion {
    PORTERO, DEFENSA, CENTROCAMPISTA, DELANTERO
}

Demarcacion delantero = Demarcacion.DELANTERO;
String nombre = delantero.name(); //Devuelve un String con el nombre de
la constante (DELANTERO)
int posicion = delantero.ordinal(); // Devuelve un entero con la posición
del enum según está declarado

Demarcacion portero = Demarcacion.PORTERO;
if(delantero.compareTo(portero) > 0) { //Devolverá un valor superior a 0,
ya que la posición de delantero es mayor que la de portero
    System.out.println("El delantero está en una posición superior al
portero");
}
else{
    System.out.println("El portero está en una posición superior al
delantero");
}

Demarcacion[] demarcaciones = Demarcacion.values(); //Devuelve un array
que contiene todos los enum

```

Los enums los podemos incluir en el fichero de otra clase, dentro o fuera de la propia clase. Si lo hacemos dentro, normalmente es para que solo esa clase pueda utilizarlo.

```

public enum Demarcacion {
    PORTERO, DEFENSA, CENTROCAMPISTA, DELANTERO
}

public class Clase{
    ...
}

```

```

public class Clase{
    private enum Demarcacion {
        PORTERO, DEFENSA, CENTROCAMPISTA, DELANTERO
    }
    ...
}

```